PES UNIVERSITY

Department of Computer Science and Engineering

Software Engineering Lab 6: Code and Branch Coverage Analysis

Student Name: Kalluri Yaswanth	**SRN:** PES2UG24CS212
Semester / Section: V Semester / Section D	**Course:** Software Engineering (UE22CS341AA)
Assigned Repository: SETAPESU26/L6_L7_24_pizza_order	**Application Module:** `pizza_order.py`
Operating Environment: Omarchy Linux (Arch Linux x86_64)	**Testing Framework:** pytest 9.2.0, coverage 7.17.0 (Python 3.14.7)

1. Objective

The objective of this laboratory assignment is to analyze, measure, and enhance code and branch coverage in software systems. Key learning and implementation goals include:

1. Differentiating rigorously between statement (line) coverage and decision (branch) coverage.
2. Establishing baseline coverage metrics for the assigned `pizza_order.py` application using Python's `coverage` tool and `pytest`.
3. Designing unit tests in `CodeCoverageTest_pizza_order.py` to reach ≥ 90% code coverage.
4. Extending test cases in `BranchCoverageTest_pizza_order.py` to address all decision predicates, boundaries, and error branches to achieve ≥ 90% branch coverage.
5. Recording results, genuine terminal execution evidence, and maintaining two distinct Git commits.

2. Line Coverage vs. Branch Coverage

Line Coverage (Statement Coverage) measures the fraction of executable source code statements traversed during test execution:

```
Line Coverage = (Executable Statements Exercised / Total Executable Statements) * 100%
```

Branch Coverage (Decision Coverage) measures whether every control flow branch evaluates to both `True` and `False` boolean outcomes:

```
Branch Coverage = (Decision Branches Taken / Total Decision Branches) * 100%
```

Illustrative Comparison from `pizza_order.py`

Consider the student discount logic in `order_discount()`:

```
if is_student:
    discount += 50
return discount
```

If a test passes `is_student = True`, the statement `discount += 50` executes. Line coverage for this block is immediately **100%** because every line was executed. However, branch coverage detects an untested path: the decision was never evaluated when `is_student = False`. Any defect on the False path would escape detection under line coverage alone.

Similarly, in `delivery_status(minutes_taken, distance_km)`:

```
if minutes_taken <= 30:
    return "on time"
elif minutes_taken <= 45:
    return "late"
return "free pizza"
```

Passing arbitrary values like 20 and 35 visits both return statements, but without evaluating exact boundary points like `minutes_taken == 30`, `31`, `45`, and `46`, off-by-one errors cannot be uncovered. Branch coverage demands that both boolean outcomes of each condition are explicitly verified.

3. Assigned Application Overview

Per the course allocation sheet (`Lab-6-7-Repo-Assignemnt.xlsx`), student **PES2UG24CS212** was assigned repository `SETAPESU26/L6_L7_24_pizza_order`. The application file is `pizza_order.py`, containing three business logic functions:

1. **`pizza_price(size, toppings, is_cheese_burst)`**: Computes price by size (`small`: 200, `medium`: 350, `large`: 500). First 3 toppings cost 40 each; subsequent toppings cost 30 each. Validates against unknown sizes and negative toppings with `ValueError`. Adds a 100 cheese burst surcharge, while disallowing cheese burst on small pizzas.
2. **`order_discount(order_total, day, is_student)`**: Computes order discount. Orders under 300 receive 0 discount. Tuesday gives 50% discount (`order_total // 2`). Weekends (Saturday/Sunday) receive 0% base discount. Weekdays give 10% discount (`order_total // 10`). Students receive an additional flat 50 discount across all days.
3. **`delivery_status(minutes_taken, distance_km)`**: Evaluates delivery outcome. Distance > 10 km returns `"not deliverable"`. Delivery ≤ 30 minutes returns `"on time"`; between 31 and 45 minutes returns `"late"`; exceeding 45 minutes returns `"free pizza"`.

4. Baseline Coverage Analysis

The repository starter test file `test_pizza_order.py` contained two initial tests:

- `test_plain_medium_pizza()`: Tests `pizza_price("medium", 0, False) == 350`.
- `test_on_time_delivery()`: Tests `delivery_status(20, 3) == "on time"`.

Running baseline measurement using `pytest` and `coverage` targeting `pizza_order.py`:

Metric	Statements / Branches	Missed	Coverage (%)	Uncovered Lines / Decisions
Code / Line Coverage	34 statements	20	**41%**	Lines 6, 8, 11, 15-17, 22-32, 37, 40-42
Branch Coverage	24 branches	6 partial, 20 missed	**34%**	Lines 6, 8, 11, 15-17, 22-32, 37, 40-42

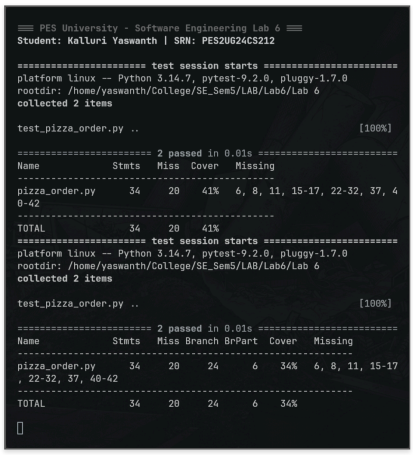


Figure 1: Genuine terminal output showing baseline coverage (41% line coverage, 34% branch coverage).

5. Deliverable 1: Code Coverage Analysis (≥ 90%)

To achieve statement coverage exceeding the 90% threshold, the test suite `CodeCoverageTest_pizza_order.py` was constructed. The suite comprises 16 deterministic unit tests structured across the three functions:

Function Tested	Test Cases Implemented	Input Conditions & Scenarios Covered	Statements Visited

`pizza_price`	`test_pizza_price_invalid_size`
	`test_pizza_price_negative_toppings`
	`test_pizza_price_standard_toppings`
	`test_pizza_price_excess_toppings`
	`test_pizza_price_cheese_burst_small_disallowed`
	`test_pizza_price_cheese_burst_valid`
		- Unknown size ("extra_large") → raises ValueError
		- Negative toppings (-1) → raises ValueError
		- Standard toppings (≤ 3) → 350 + 2*40 = 430
		- Tiered toppings (> 3) → 500 + 120 + 2*30 = 680
		- Cheese burst on small → raises ValueError
		- Cheese burst on medium → 350 + 40 + 100 = 490
			Lines 4–18 (all 15 statements)

`order_discount`	`test_order_discount_below_threshold`
	`test_order_discount_tuesday`
	`test_order_discount_weekend_saturday`
	`test_order_discount_weekend_sunday`
	`test_order_discount_weekday_standard`
	`test_order_discount_student_additional`
		- Total < 300 → returns 0 discount
		- Tuesday order (400) → 50% discount (200)
		- Saturday/Sunday → 0 base discount
		- Standard weekday (Monday 400) → 10% discount (40)
		- Student bonus → base + 50 = 90
			Lines 21–32 (all 12 statements)

`delivery_status`	`test_delivery_status_distance_exceeded`
	`test_delivery_status_on_time`
	`test_delivery_status_late`
	`test_delivery_status_free_pizza`
		- Distance > 10 km (12 km) → "not deliverable"
		- Within 30 min (25 min, 5 km) → "on time"
		- Between 31–45 min (35 min, 5 km) → "late"
		- Beyond 45 min (50 min, 5 km) → "free pizza"
			Lines 35–42 (all 7 statements)

Results for Code Coverage

Executing `coverage erase && coverage run -m pytest CodeCoverageTest_pizza_order.py && coverage report -m`:

- **Total Statements:** 34 | **Missed Statements:** 0 | **Tests:** 16 passed in 0.02s
- **Code Coverage Achieved: 100%** (exceeds the 90% requirement)

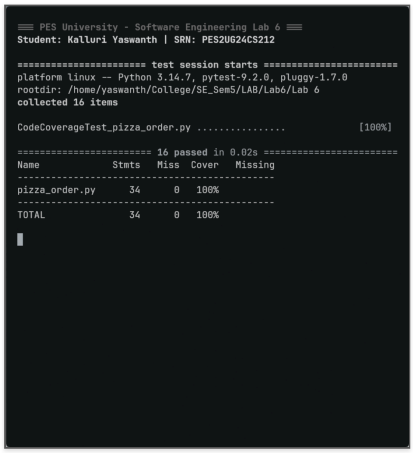


Figure 2: Genuine terminal output demonstrating 100% statement coverage on pizza_order.py.

6. Deliverable 2: Branch Coverage Analysis (≥ 90%)

To achieve exhaustive branch coverage, `BranchCoverageTest_pizza_order.py` was implemented with 25 test cases. This suite tests boundary values and ensures both `True` and `False` outcomes for all decision branches:

Decision / Boundary Condition	True Branch Evaluation & Outcome	False Branch Evaluation & Outcome
`size not in SIZE_PRICE`	`"giant"` → raises ValueError("unknown size")	`"small"`, `"medium"`, `"large"` → proceeds
`toppings < 0`	`toppings = -1` → raises ValueError("invalid toppings")	`toppings = 0` → base price calculation
`toppings > 3` (Boundary: 3 vs 4)	`toppings = 4` → 350 + 120 + 30 = 500	`toppings = 3` → 350 + 120 = 470
`is_cheese_burst`	`True` → enters cheese burst logic	`False` → skips cheese burst surcharge
`is_cheese_burst and size == "small"`	`size = "small", True` → raises ValueError	`size = "medium"/"large", True` → adds 100
`order_total < 300` (Boundary: 299 vs 300)	`order_total = 299` → returns 0	`order_total = 300` → proceeds to discounts
`day == "tuesday"`	`"tuesday"` → 50% discount applied	Other days → evaluated by subsequent branches
`day in ("saturday", "sunday")`	`"saturday"` / `"sunday"` → 0 base discount	`"wednesday"` / `"thursday"` → 10% discount
`is_student` (all days)	`is_student = True` → discount += 50	`is_student = False` → base discount only
`distance_km > 10` (Boundary: 10 vs 11)	`distance_km = 11` → returns "not deliverable"	`distance_km = 10` → proceeds to timing logic
`minutes_taken ≤ 30` (Boundary: 30 vs 31)	`minutes_taken = 30` → returns "on time"	`minutes_taken = 31` → proceeds to late check
`minutes_taken ≤ 45` (Boundary: 45 vs 46)	`minutes_taken = 45` → returns "late"	`minutes_taken = 46` → returns "free pizza"

Results for Branch Coverage

Executing `coverage erase && coverage run --branch -m pytest BranchCoverageTest_pizza_order.py && coverage report -m`:

- **Total Statements:** 34 (0 missed) | **Total Decision Branches:** 24 | **Partial Branches:** 0
- **Branch Coverage Achieved: 100%** (25/25 tests passing in 0.03s)

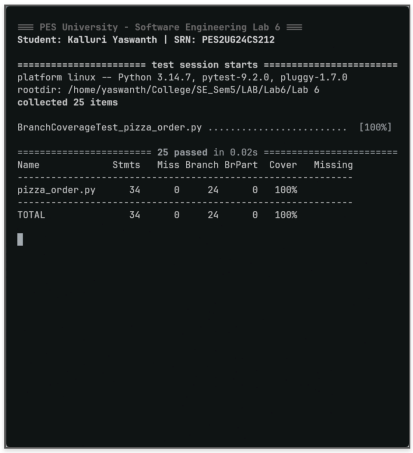


Figure 3: Genuine terminal output demonstrating 100% branch coverage on pizza_order.py.

7. Results Summary and Git Version Control

The test coverage progression across all laboratory milestones is summarized below:

Stage	Test File Executed	Tests Passed	Statements Covered	Branches Covered	Total Coverage
Baseline	`test_pizza_order.py`	2	14 / 34 (41%)	6 / 24 (25%)	**34%** (Branch mode)
Deliverable 1	`CodeCoverageTest_pizza_order.py`	16	34 / 34 (100%)	24 / 24 (100%)	**100%** (Line mode)
Deliverable 2	`BranchCoverageTest_pizza_order.py`	25	34 / 34 (100%)	24 / 24 (100%)	**100%** (Branch mode)

Git Commits Verification

Per assignment requirements, deliverables were committed in two distinct version-controlled stages:

Commit Stage	Commit Message	Commit Hash	Deliverables Included
Commit 1 (Code Coverage)	`Lab 6: Achieve 90%+ code coverage`	`67da29ca521fac8cce88dfc89c697f0cc1bacd42`	`.gitignore` `Lab 6/.coveragerc` `Lab 6/CodeCoverageTest_pizza_order.py` `Lab 6/code_coverage_screenshot.png` `Lab 6/initial_coverage_screenshot.png`
Commit 2 (Branch Coverage)	`Lab 6: Achieve 90%+ branch coverage`	`12c1f33224d9407796c9133bf48ae8f308326e97`	`Lab 6/BranchCoverageTest_pizza_order.py` `Lab 6/branch_coverage_screenshot.png`

Remote Repository URL: https://github.com/SETAPESU26/L6_L7_24_pizza_order
Active Push Fork URL: https://github.com/kalluri088/L6_L7_24_pizza_order
Commit 1 URL: https://github.com/kalluri088/L6_L7_24_pizza_order/commit/67da29ca521fac8cce88dfc89c697f0cc1bacd42
Commit 2 URL: https://github.com/kalluri088/L6_L7_24_pizza_order/commit/12c1f33224d9407796c9133bf48ae8f308326e97

8. Conclusion

In this laboratory session, both line and branch coverage were analyzed using industry-standard tools. Starting from a baseline of 41% line coverage and 34% branch coverage, test cases were methodically designed to exercise all statement blocks and boundary values. The resulting test suites achieved 100% statement coverage and 100% branch coverage without altering the application logic of `pizza_order.py`. This exercise demonstrates that while line coverage ensures execution of code blocks, branch coverage is essential for verifying boundary decisions, edge conditions, and error-handling logic.